

CODING SKILL TREE

Dispensa progressiva di algoritmi e complessità

Dalle implementazioni non conformi agli invarianti efficienti

Manuale cumulativo per il ripasso offline dei pattern algoritmici emersi nelle quest settimanali e nelle relative review tecniche.

Giuseppe Sinatra

Edizione progressiva - aggiornata al 12 agosto 2026

Nota metodologica

Questa dispensa non è una raccolta generale di algoritmi. Ogni capitolo nasce da un'evidenza osservabile: un pattern nuovo affrontato in assessment, una soluzione non corretta oppure un'implementazione corretta sugli esempi ma non conforme ai vincoli di scala.

L'analisi segue una struttura costante: modello del problema, approccio iniziale, diagnosi, invariante dell'algoritmo efficiente, argomento di correttezza, complessità ed elementi utili al riconoscimento del pattern. I riquadri rossi conservano il valore diagnostico dell'implementazione iniziale; i riquadri verdi presentano la versione di riferimento validata in review.

Regola di lettura

Il codice rosso non è necessariamente errato negli output. Può essere semanticamente corretto e, al tempo stesso, inaccettabile rispetto ai limiti computazionali del problema. La distinzione tra correttezza funzionale ed efficienza asintotica è parte integrante della valutazione algoritmica.

Indice

Nota metodologica	ii
1 Predicati esatti e sliding window	1
1.1 Predicati esatti nell'aggregazione di record	1
1.1.1 Modello, ipotesi e notazione	1
1.1.2 Approccio iniziale ed errore logico	1
1.1.3 Algoritmo corretto e invariante	2
1.1.4 Correttezza, complessità ed edge case	2
1.2 Sliding window: modello del problema	3
1.3 Sliding window: approccio iniziale	4
1.3.1 Diagnosi tecnica	4
1.4 Sliding window: algoritmo efficiente	5
1.5 Sliding window: argomento di correttezza	5
1.6 Sliding window: analisi di complessità	6
1.7 Sliding window: edge case rilevanti	6
1.8 Sliding window: come riconoscere il pattern	6
1.9 Checklist di ripasso	7
2 Stato incrementale	8
2.1 Principio comune: conservare lo stato	8
2.2 Sliding window con vincolo sulla somma	8
2.2.1 Modello e precondizioni	8
2.2.2 Submission originale e diagnosi	9
2.2.3 Algoritmo lineare e invariante	10
2.2.4 Argomento di correttezza	10
2.3 Query online sul numero di modelli attivi	11
2.3.1 Modello del registro	11

2.3.2	Submission originale e costo delle query	11
2.3.3	Aggregato incrementale e transizioni di soglia	12
2.3.4	Correttezza e complessità	13
2.4	Segnali di riconoscimento ed edge case	13
Indice dei pattern		14

Predicati esatti e sliding window

Origine: Week 001, esercizi 2 e 4

Contesto: aggregazione di record e picco di richieste temporali

Esito della review: un errore logico e un vincolo di scala non rispettato

Pattern: predicati esatti; sliding window a due indici

1.1 Predicati esatti nell'aggregazione di record

1.1.1 Modello, ipotesi e notazione

Sia $R = (r_0, \dots, r_{n-1})$ una sequenza di record. Ogni record può contenere i campi `model`, `tokens` e `success`. Per ogni nome di modello valido occorre calcolare numero di run, numero di successi e somma dei token. La condizione sui successi è intenzionalmente stretta:

$$\text{successo}(r_i) \iff r_i[\text{success}] \text{ is True.}$$

Il contratto non afferma che il campo contenga esclusivamente booleani; pertanto non è lecito sostituire questa condizione con la truthiness.

1.1.2 Approccio iniziale ed errore logico

La submission costruisce correttamente un dizionario indicizzato per modello, gestisce token mancanti o nulli e non modifica gli input. La riga che calcola `successful_run`, tuttavia, accetta ogni oggetto truthy. Ad esempio, 1 e "yes" aumentano il contatore pur non essendo l'oggetto booleano `True`. Si tratta di un errore di correttezza, non di una semplice inefficienza.

Il codice seguente conserva integralmente la submission; sono omessi soltanto gli spazi finali invisibili, privi di effetto sull'esecuzione, per esigenze di presentazione tipografica.

Submission originale - errore logico sul predicato

```
1 def summarize_model_runs(runs):
2     """Aggrega run, successi e token per modello."""
3     lookup = {}
4     for record_or in runs:
5         record=record_or.copy()
6         model=record.get('model')
7         tokens= 0 if record.get('tokens') is None else record.get('tokens')
8         successful_run=1 if record.get("success") else 0
9         if model is None:
```

```

10         continue
11     if model not in lookup:
12         lookup[model]={"total_runs": 1, "successful_runs": successful_run, "
total_tokens": tokens}
13     else:
14         lookup[model]["total_runs"]+=1
15         lookup[model]["successful_runs"]+=successful_run
16         lookup[model]["total_tokens"]+=tokens
17
18     return lookup

```

1.1.3 Algoritmo corretto e invariante

La struttura di aggregazione resta appropriata. È sufficiente inizializzare le statistiche del modello, aggiornare esattamente una volta il numero di run e tradurre letteralmente il predicato con `is True`.

Invariante dell'aggregazione

Dopo avere elaborato i primi i record, per ogni modello m presente nel dizionario le tre statistiche sono esattamente quelle dei record validi tra r_0 e r_{i-1} con modello m ; in particolare, `successful_runs` conta soltanto i record il cui valore di `success` è l'oggetto `True`.

Implementazione di riferimento - predicato conforme

```

1 def summarize_model_runs(runs):
2     """Aggrega run, successi e token per modello."""
3     summary = {}
4     for record in runs:
5         model = record.get("model")
6         if model is None:
7             continue
8
9         if model not in summary:
10            summary[model] = {
11                "total_runs": 0,
12                "successful_runs": 0,
13                "total_tokens": 0,
14            }
15
16            stats = summary[model]
17            stats["total_runs"] += 1
18            stats["successful_runs"] += int(record.get("success") is True)
19            tokens = record.get("tokens")
20            stats["total_tokens"] += 0 if tokens is None else tokens
21
22    return summary

```

1.1.4 Correttezza, complessità ed edge case

L'invariante vale inizialmente perché il dizionario è vuoto. Ogni record senza modello valido viene ignorato come richiesto. Per un record valido, l'algoritmo aggiorna una sola voce: incrementa il

numero di run, aggiunge un successo se e solo se il predicato esatto è vero e somma zero oppure il numero di token. L'invariante è quindi preservato; dopo n iterazioni descrive l'intero input e implica la correttezza del risultato.

Il tempo è $O(n)$ in media, assumendo accessi hash $O(1)$, e lo spazio ausiliario è $O(k)$ per k modelli distinti. I casi da verificare includono lista vuota, modello mancante o `None`, nome di modello vuoto, token mancanti, nulli o uguali a zero, e valori `success` quali `False`, `None`, `0`, `1` e stringhe non vuote.

Segnale di riconoscimento

Espressioni come “soltanto quando”, “esattamente”, “è `True`” o “è `None`” richiedono di distinguere identità, uguaglianza e truthiness. Prima di codificare, trascrivere il predicato della specifica in una condizione booleana senza allargarne il dominio.

1.2 Sliding window: modello del problema

Sia

$$T = (t_0, t_1, \dots, t_{n-1})$$

una sequenza di timestamp ordinata in modo non decrescente e sia $w > 0$ la durata della finestra. Si vuole determinare il massimo numero di eventi contenuti in un intervallo semiaperto di ampiezza w :

$$M = \max_s |\{t_i \in T \mid s \leq t_i < s + w\}|.$$

La scelta dell'intervallo semiaperto è sostanziale: un evento posto esattamente in $s + w$ non appartiene alla finestra che inizia in s .

Precondizioni operative

La soluzione lineare presuppone timestamp già ordinati. Se l'ordinamento non è garantito dal contratto, occorre prima ordinare la sequenza, portando il costo complessivo a $O(n \log n)$. La durata deve essere strettamente positiva; in caso contrario la funzione solleva `ValueError`.

1.3 Sliding window: approccio iniziale

L'implementazione presentata durante la quest seleziona ogni timestamp come possibile inizio e, per ciascuno, visita nuovamente l'intera sequenza. Questa strategia è concettualmente semplice e restituisce gli output attesi, ma esegue fino a n^2 verifiche di appartenenza.

Implementazione iniziale - non conforme al vincolo di scala

```
1 def max_requests_in_window(timestamps, window_seconds):
2     """Restituisce il picco di richieste in una finestra temporale."""
3     count_max=0
4     if window_seconds <= 0:
5         raise ValueError("La finestra dev'essere un valore positivo")
6     for start in timestamps:
7         count=0
8         for input in timestamps:
9             if start <= input < start+window_seconds:
10                 count+=1
11             count_max=max(count, count_max)
12
13     return count_max
```

1.3.1 Diagnosi tecnica

Il problema non risiede nella condizione di appartenenza, che rappresenta correttamente l'intervallo semiaperto. L'inefficienza nasce dalla perdita di informazione tra due finestre consecutive: quando l'estremo destro avanza, gli eventi ancora validi vengono contati nuovamente da zero.

Inoltre, il nome locale `input` oscura la funzione built-in omonima di Python. Non altera il risultato in questo caso, ma riduce la chiarezza del codice e può generare errori in estensioni successive.

Con un limite di $n = 200\,000$, una crescita quadratica è incompatibile con un coding assessment ordinario:

$$n^2 = 40\,000\,000\,000$$

controlli potenziali. Il superamento dei test visibili certifica soltanto alcuni risultati, non il rispetto dei vincoli computazionali.

1.4 Sliding window: algoritmo efficiente

Poiché i timestamp sono ordinati, gli eventi validi per un dato estremo destro formano sempre un segmento contiguo della sequenza. È quindi sufficiente mantenere due indici:

- **right** introduce il nuovo evento corrente;
- **left** avanza soltanto quando l'evento più vecchio non rientra più nella finestra.

Invariante della finestra

Dopo il ciclo **while**, tutti e soli gli eventi con indice compreso tra **left** e **right** appartengono alla finestra semiaperta che termina sul timestamp corrente. In particolare,

$$t_{\text{right}} - t_{\text{left}} < w.$$

L'indice **left** è il più piccolo indice che rende vera questa disuguaglianza.

Implementazione di riferimento - complessità lineare

```
1 def max_requests_in_window(timestamps, window_seconds):
2     """Restituisce il picco di richieste in una finestra temporale."""
3     if window_seconds <= 0:
4         raise ValueError("La finestra dev'essere un valore positivo")
5
6     left = 0
7     best = 0
8
9     for right, current_time in enumerate(timestamps):
10        while current_time >= timestamps[left] + window_seconds:
11            left += 1
12
13        current_count = right - left + 1
14        best = max(best, current_count)
15
16    return best
```

1.5 Sliding window: argomento di correttezza

Si consideri un indice destro fissato. Prima dell'esecuzione del **while**, la finestra può contenere timestamp troppo vecchi. Ogni iterazione incrementa **left** finché il timestamp corrente non dista meno di w dal timestamp più vecchio conservato. Per ordinamento, tutti gli elementi successivi a **left** soddisfano allora la stessa condizione.

Alla terminazione del ciclo, **left** è minimo: se esistesse un indice valido più piccolo, il **while** si sarebbe fermato prima. Di conseguenza, $\text{right} - \text{left} + 1$ è il massimo numero di eventi in una finestra valida il cui evento più recente è t_{right} . Considerando in successione ogni possibile estremo destro e conservando il massimo, l'algoritmo restituisce M .

1.6 Sliding window: analisi di complessità

Strategia	Tempo	Spazio ausiliario
Scansione annidata	$O(n^2)$	$O(1)$
Sliding window	$O(n)$	$O(1)$
Ordinamento + sliding window	$O(n \log n)$	dipende dall'ordinamento

Il ciclo `while` annidato non rende quadratica la seconda soluzione. L'indice `left` non arretra mai: nel corso dell'intera esecuzione può avanzare al massimo n volte. Anche `right` percorre la sequenza una sola volta; il lavoro totale è quindi lineare.

1.7 Sliding window: edge case rilevanti

- **Sequenza vuota:** il ciclo non viene eseguito e il risultato è zero.
- **Timestamp duplicati:** restano simultaneamente nella finestra e vengono conteggiati come eventi distinti.
- **Evento sul bordo destro:** se la distanza è esattamente w , l'evento più vecchio viene escluso, coerentemente con $[s, s + w)$.
- **Durata non positiva:** il problema non definisce una finestra valida; la funzione segnala esplicitamente l'errore.
- **Input non ordinato:** l'invariante non è garantito; occorre ordinare oppure rifiutare l'input secondo il contratto.

1.8 Sliding window: come riconoscere il pattern

La sliding window è una candidata naturale quando il problema presenta contemporaneamente:

- una sequenza ordinata o ordinabile;
- un intervallo contiguo definito da una soglia;
- una statistica da massimizzare o minimizzare su tutti gli intervalli;
- finestre consecutive che condividono gran parte degli elementi.

Regola operativa per l'assessment

Quando due finestre consecutive differiscono per pochi elementi, chiedersi quale stato possa essere aggiornato incrementalmente. Se ogni elemento entra ed esce dalla struttura al massimo una volta, è spesso possibile ottenere una soluzione $O(n)$.

1.9 Checklist di ripasso

Prima di considerare concluso un problema analogo, verificare:

1. se l'ordinamento è garantito oppure deve essere costruito;
2. se gli estremi dell'intervallo sono inclusivi o esclusivi;
3. quale invariante deve valere dopo l'avanzamento di `left`;
4. quante volte ciascun indice può avanzare complessivamente;
5. se la complessità ottenuta è compatibile con la dimensione massima dichiarata.

Stato di apprendimento

Il pattern è stato introdotto e analizzato, ma non è ancora masterizzato. La mastery richiederà implementazioni lineari autonome, corrette e completate nel timebox su più problemi con formulazioni differenti.

Stato incrementale e lavoro riutilizzabile

Origine: Week 002, esercizi 3 e 4

Contesto: batch contigui entro un budget e registro dinamico di modelli

Esito della review: output corretti, vincoli di scala non rispettati

Pattern: sliding window con somma corrente; query online incrementali

2.1 Principio comune: conservare lo stato

I due problemi della settimana hanno formulazioni differenti, ma condividono la stessa struttura algoritmica. Una sequenza di operazioni consecutive modifica soltanto una piccola parte dello stato precedente:

- nella finestra sul budget entra un costo dall'estremo destro e possono uscire alcuni costi dall'estremo sinistro;
- nel registro cambia la frequenza di un solo modello per operazione.

Ricalcolare ogni volta l'intera finestra o l'intero registro produce risultati corretti, ma perde l'informazione già disponibile. L'obiettivo dell'approccio incrementale è mantenere un riassunto sufficiente dello stato e aggiornarlo soltanto in corrispondenza della modifica corrente.

Domanda diagnostica

Se due iterazioni consecutive condividono quasi tutti i dati, quale quantità può essere aggiornata aggiungendo ciò che entra e rimuovendo ciò che esce? Questa domanda separa spesso una soluzione quadratica da una lineare.

2.2 Sliding window con vincolo sulla somma

2.2.1 Modello e precondizioni

Sia $C = (c_0, \dots, c_{n-1})$ una sequenza di costi non negativi e sia $B \geq 0$ il budget. Si cerca la massima lunghezza di un segmento contiguo $C[l : r]$ tale che

$$\sum_{i=l}^r c_i \leq B.$$

La non negatività è una preconditione essenziale. Se la somma supera il budget, estendere ulteriormente la finestra non può ridurla; per renderla di nuovo valida occorre avanzare l'estremo sinistro. Con valori negativi questa monotonia non sarebbe garantita e servirebbe un algoritmo diverso.

2.2.2 Submission originale e diagnosi

La submission seguente è funzionalmente corretta sugli input ammessi. La variabile `i` seleziona ogni possibile inizio e `j` estende il segmento finché il budget è rispettato. Il problema è che `j` e `tot_budget` vengono reinizializzati per ogni nuovo `i`; i costi già sommati vengono quindi visitati molte volte.

Il codice conserva integralmente la submission; nella sola stringa d'errore, `può` è reso come `puo'` per compatibilità tipografica con il motore dei riquadri. La modifica non ha alcun effetto sulla logica.

Submission originale - output corretti, costo quadratico

```
1 def longest_batch_within_budget(costs, budget):
2     """Restituisce la massima lunghezza contigua entro il budget."""
3     len_series=0
4     if budget <0:
5         raise ValueError("Il budget non puo' essere negativo")
6
7     for i in range(len(costs)):
8         j=0
9         left=costs[i]
10        tot_budget=left
11        n_serie=0
12        while tot_budget<=budget:
13            n_serie+=1
14            j+=1
15            if i+j<len(costs):
16                tot_budget+=costs[i+j]
17            else:
18                break
19        len_series =max(len_series, n_serie)
20    return len_series
```

Nel caso peggiore, ad esempio con tutti i costi uguali a zero, il numero di visite è

$$n + (n - 1) + \dots + 1 = \frac{n(n + 1)}{2} = \Theta(n^2).$$

L'espressione `i+j` calcola un indice in tempo costante, ma non riduce il numero complessivo di elementi riesaminati. Nei benchmark della review, il passaggio da 4.000 a 8.000 elementi ha moltiplicato il tempo per circa quattro, coerentemente con la crescita quadratica.

2.2.3 Algoritmo lineare e invariante

La finestra efficiente mantiene gli indici `left` e `right` e una somma corrente. Quando `right` avanza, il nuovo costo viene aggiunto. Se la somma supera il budget, si sottraggono in ordine i costi che escono da sinistra.

Invariante della somma corrente

Al termine del ciclo `while`, vale

$$\text{current_sum} = \sum_{i=\text{left}}^{\text{right}} c_i \leq B,$$

e `left` è il più piccolo indice che rende valida la finestra con estremo destro `right`.

Implementazione di riferimento - sliding window $O(n)$

```

1 def longest_batch_within_budget(costs, budget):
2     """Restituisce la massima lunghezza contigua entro il budget."""
3     if budget < 0:
4         raise ValueError("Il budget non puo' essere negativo")
5
6     left = 0
7     current_sum = 0
8     best = 0
9
10    for right, cost in enumerate(costs):
11        current_sum += cost
12
13        while current_sum > budget:
14            current_sum -= costs[left]
15            left += 1
16
17        best = max(best, right - left + 1)
18
19    return best

```

2.2.4 Argomento di correttezza

Prima di elaborare un elemento la somma descrive esattamente la finestra precedente. L'aggiunta di c_{right} estende tale finestra di un elemento. Se il budget è superato, ogni iterazione del `while` sottrae esattamente il costo escluso e avanza `left`; l'uguaglianza tra somma memorizzata e somma del segmento è quindi preservata.

Quando il ciclo termina, la finestra è valida. Inoltre `left` è minimo: il costo precedente è stato rimosso soltanto mentre la somma superava B . La lunghezza `right-left+1` è pertanto la massima lunghezza valida con quell'estremo destro. Considerando tutti gli estremi destri e conservando il massimo si ottiene la soluzione globale.

Ogni elemento entra una volta tramite **right** ed esce al massimo una volta tramite **left**. Il tempo è $O(n)$ e lo spazio ausiliario è $O(1)$.

2.3 Query online sul numero di modelli attivi

2.3.1 Modello del registro

Sia $f(m)$ il numero corrente di deployment del modello m . Le operazioni possono incrementare o decrementare una frequenza, richiederne il valore o chiedere il numero di modelli distinti attivi:

$$A = |\{m \mid f(m) > 0\}|.$$

Una query è detta *online* quando deve essere risolta nello stato prodotto da tutte le operazioni precedenti, senza conoscere o riorganizzare quelle successive.

2.3.2 Submission originale e costo delle query

La submission gestisce correttamente registrazioni, ritiri, conteggi e transizioni a zero. La query **distinct**, tuttavia, visita ogni chiave del dizionario per ricostruire A .

Submission originale - distinct ricalcolato integralmente

```

1 def process_model_registry(initial_models, operations):
2     """Elabora aggiornamenti e query sul registro dei modelli."""
3     result=[]
4     registry_model={}
5     for model in initial_models:
6         if model not in registry_model:
7             registry_model[model]=0
8             registry_model[model]+=1
9
10    for command in operations:
11        if command[0]=="count":
12            model=command[1]
13            count=registry_model.get(model,0)
14            result.append(count)
15            continue
16        if command[0]=="distinct":
17            count=0
18            for model in registry_model:
19                if registry_model.get(model)!=0:
20                    count+=1
21            result.append(count)
22            continue
23        if command[0]=="retire":
24            model=command[1]
25            if model in registry_model and registry_model.get(model)>0:
26                registry_model[model]-=1
27            continue
28        if command[0]=="register":
29            model=command[1]
30            registry_model[model]=registry_model.get(model,0)+1
31            continue
32    return result

```

Se il registro contiene m chiavi e si eseguono q query `distinct`, il costo può raggiungere $O(qm)$. Con m e q entrambi proporzionali alla dimensione dell'input, il comportamento è quadratico.

2.3.3 Aggregato incrementale e transizioni di soglia

Una singola operazione modifica la frequenza di un solo modello. Il valore A non cambia per ogni incremento o decremento, ma soltanto quando una frequenza attraversa la soglia zero:

- $0 \rightarrow 1$: un modello diventa attivo, quindi A aumenta;
- $1 \rightarrow 0$: un modello cessa di essere attivo, quindi A diminuisce;
- $k \rightarrow k + 1$ con $k > 0$ e $k \rightarrow k - 1$ con $k > 1$: A resta invariato.

Invariante del registro

Dopo ogni operazione, `active_models` è uguale al numero di chiavi del dizionario la cui frequenza è strettamente positiva. La query `distinct` può quindi rispondere in $O(1)$.

Implementazione di riferimento - query tutte $O(1)$ in media

```

1 def process_model_registry(initial_models, operations):
2     """Elabora aggiornamenti e query sul registro dei modelli."""
3     frequencies = {}
4     active_models = 0
5     results = []
6
7     for model in initial_models:
8         current = frequencies.get(model, 0)
9         if current == 0:
10             active_models += 1
11             frequencies[model] = current + 1
12
13     for operation in operations:
14         command = operation[0]
15
16         if command == "count":
17             results.append(frequencies.get(operation[1], 0))
18
19         elif command == "distinct":
20             results.append(active_models)
21
22         elif command == "register":
23             model = operation[1]
24             current = frequencies.get(model, 0)
25             if current == 0:
26                 active_models += 1
27                 frequencies[model] = current + 1
28
29         elif command == "retire":
30             model = operation[1]
31             current = frequencies.get(model, 0)
32             if current > 0:
33                 frequencies[model] = current - 1
34                 if current == 1:
35                     active_models -= 1

```



```
36
37     return results
```

2.3.4 Correttezza e complessità

Durante l'inizializzazione, `active_models` viene incrementato esattamente alla prima occorrenza di ogni modello, quindi l'invariante vale prima delle query. Un'operazione `count` o `distinct` non cambia lo stato. Una registrazione modifica A soltanto se il conteggio precedente è zero; un ritiro valido lo modifica soltanto se il conteggio precedente è uno. In ogni caso l'invariante è preservato per induzione.

Assumendo accessi hash $O(1)$ in media, inizializzazione e operazioni richiedono tempo $O(n + q)$, dove n è il numero di deployment iniziali e q il numero di operazioni. Lo spazio è $O(m + r)$: m modelli distinti memorizzati e r risposte prodotte.

2.4 Segnali di riconoscimento ed edge case

- **Finestra contigua e valori non negativi:** una soglia sulla somma suggerisce una finestra espandibile e restringibile.
- **Query ripetute su un aggregato:** se ogni update tocca un solo elemento, verificare se l'aggregato può essere aggiornato localmente.
- **Soglie di stato:** conteggi di elementi attivi, non vuoti o disponibili cambiano spesso soltanto attraversando zero.
- **Sequenza vuota:** la finestra restituisce zero; il registro parte con zero modelli attivi.
- **Ritiro di un elemento assente:** non cambia frequenze né aggregato.
- **Costi uguali a zero:** possono restare tutti nella finestra e devono contribuire alla lunghezza.

Stato di apprendimento

I due invarianti sono stati analizzati e compresi durante la review, ma non sono ancora masterizzati. La mastery richiede implementazioni autonome, lineari e completate entro il timebox su formulazioni differenti.

Indice dei pattern

Pattern	Prima introduzione	Stato al capitolo
Predicati esatti	Week 001	Introdotta, da consolidare
Sliding window a due indici	Week 001	Riconosciuto; autonomia da verificare
Query online incrementali	Week 002	Introdotta, da consolidare